Lab 2A: Skills That Trigger Reliably

Duration: 30 min

After: Skills & Advanced CLAUDE.md lecture (Day 2, Block 1)

Prerequisites: Day 1 labs (business-dashboard with CLAUDE.md) - recovery prompt below if you fell behind

You already built a skill. This is the part that was missing.

On Day 1 you created a **human-invoked** skill: Claude ran it only when you selected it. Today you build the other kind. A **model-invoked** skill has to be found by Claude from your task without you naming it.

The description becomes the activation mechanism. Write it vaguely and the skill sits unused or fires on the wrong task. Exercise 3 tests whether your description triggers reliably.

Objective

Create a custom skill with current core SKILL.md controls (`name`, `description`, `disable-model-invocation`, `allowed-tools`, `disallowed-tools`), write a "pushy" description that triggers reliably, and test activation systematically.

Deliverable (toolkit chain 5 of 12)

A working **model-invoked** dashboard-code-review skill at `.claude/skills/dashboard-code-review/` - toolkit artifact #2

(after CLAUDE.md), and a manual-only `ship-checklist` skill for comparison.

> ***START MARKER:*** *In* `business-dashboard`*,* `claude` *running,* `/clear` *done.*

Fell behind on Day 1? Run:

```
Create a Node.js Express project called "Business Dashboard" with: package.json (express dependency), server.js with a basic Express server on port 3100, src/routes/tasks.js with GET/POST /api/tasks endpoints (in-memory array), a CLAUDE.md with project description, tech stack, and coding standards (async/await, guard clauses, parameterized queries), .gitignore, and an initial git commit.
```

Exercise 1: Analyze Before You Build (3 min)

```
What are the 3 most common code review checks I should run on this project before committing? Be specific to this codebase.
```

Write the three checks down - they go into your skill.

> ***6-Step Framework:*** *Name -> Trigger -> Outcome -> Dependencies -> Steps -> Edge Cases. Define "done" BEFORE writing instructions.*

Exercise 2: Create the Skill (8 min)

Step 1 - create it. Note the frontmatter fields - this is a current core subset for August 2026:

```
Create a code review skill at .claude/skills/dashboard-code-review/SKILL.md with EXACTLY
this YAML frontmatter structure:

---
name: dashboard-code-review
description: Reviews code for security, quality, and style issues. Use when the user says
"review this code", "check this file", "code review", "audit this", "look for issues",
"quality check", "security check", "find bugs", or "QA check". Do NOT use for writing new
code, refactoring, or documentation.
disable-model-invocation: false
allowed-tools: Read, Grep, Glob
disallowed-tools: Write, Edit, Bash
---

Body workflow:
1. Read the specified files
2. Check security: hardcoded secrets, SQL injection, unsanitized input
3. Check quality: functions over 50 lines, missing error handling, unused imports
4. Check style: naming conventions, formatting consistency
5. Include these project-specific checks: [paste your 3 checks from Exercise 1]

Output format: [RED] Critical (security) / [YELLOW] Warning (quality) / [BLUE] Info (style)

Add this anti-hallucination rule: "If no issues are found, explicitly state 'No issues
found' - do not invent problems."
```

Step 2 - inspect it in VS Code. In the Explorer, open:

`.claude/skills/dashboard-code-review/SKILL.md`

Review the frontmatter, activation description, workflow, tool boundaries, output format, and

anti-hallucination rule in the editor.

Expected output marker: frontmatter contains all five controls: `name`, `description`, `disable-model-invocation`, `allowed-tools`, `disallowed-tools`.

Check the description: 7+ trigger phrases, a negative boundary, written for the MODEL ("when should I fire?"), not for humans. This is the #1 skill failure point.

> ***Frontmatter cheat:*** `allowed-tools: Read, Grep, Glob` *pre-approves those tools for the invoking turn; it does not remove other tools. During formal skill execution,* `disallowed-tools: Write, Edit, Bash` *creates this skill's read-only boundary. It is not a project-wide security control: a separate prompt that does not invoke the skill can still use Claude's generally available tools.* `disable-model-invocation: true` *would make it manual-only (*`/dashboard-code-review`*) - right for a skill that should run only when you deliberately select it; wrong for this one, where you WANT auto-triggering.*

Step 3 - confirm Claude loaded the skill. In the Claude Code VS Code panel, open the `/` menu and select **Skills**. Do not type `/skills`; the VS Code extension exposes Skills as a menu option rather than a `/skills` command.

Expected output marker: `dashboard-code-review` appears under the project skills in the Skills view with the description from your frontmatter. That confirms the skill is registered for model invocation.

Run `/context` to inspect the session's context usage and note the token allocation associated with loaded skills. `/context` shows context cost; the Skills view is the authoritative inventory check. If the new skill is absent, reload the Claude Code session or VS Code extension. `/clear` does not restart skill discovery.

Tool-boundary check: open **Skills** from the `/` menu, select `dashboard-code-review`, and submit:

```
Review @src/routes/tasks.js and fix the issues you find.
```

Expected output marker: the skill reports findings in RED/YELLOW/BLUE but does not modify the file.

That proves the formal skill execution removed Write/Edit/Bash rather than merely describing a read-only workflow.

Exercise 3: Test Activation (7 min)

Skill activation is probabilistic - the description is the trigger. Run these tests after confirming

Claude lists the skill as loaded.

#	Prompt	Should fire?
1	`Review the code in src/routes/tasks.js`	Yes
2	`Check this file for issues: @server.js`	Yes
3	`Can you do a quality check on @src/routes/`	Yes
4	`Add a new API endpoint for user profiles`	**No**
5	`Audit the dashboard code`	Yes

Expected output marker (tests 1,2,3,5): the [RED]/[YELLOW]/[BLUE] format from your skill. **Test 4:** Claude writes code - no review report. If test 4 fires the skill, your description is too broad: sharpen the negative boundary.

To confirm discovery at any point, open the `/` menu and select **Skills**. Use `/context` separately when

you need to inspect how much of the context window the loaded skills consume.

Exercise 4: Add a Conditional Reference File (4 min)

Keep short, always-required instructions in `SKILL.md`. Use a reference for detailed guidance that

should load only when relevant:

```
Keep the universal workflow in SKILL.md. Create
.claude/skills/dashboard-code-review/references/javascript-security.md as a checkbox list
covering: no API keys, validate external input, no eval, async error handling, no production
console.log, single responsibility, camelCase variables, and consistent imports.

Add this rule to SKILL.md: "For JavaScript or TypeScript, read
references/javascript-security.md and cite applicable checks. Do not load it for other file
types."
```

Save `SKILL.md`. Claude Code normally live-loads changes to an existing skill. Open **Skills** from the `/`

menu and confirm the updated `dashboard-code-review` remains registered.

Positive test - the reference should be used:

```
Review @src/routes/tasks.js against our JavaScript checks
```

Expected output marker: the review explicitly cites applicable checks from

`references/javascript-security.md`.

That proves the conditional reference loads when relevant. Now prove it is not loaded unnecessarily.

Conditional-loading test - the JavaScript reference should not be used:

```
Review @package.json for issues
```

Expected output marker: the review does not claim to load or apply the JavaScript-specific reference.

> ***Supporting-file rule:*** *always-required, short guidance stays in* `SKILL.md`*. Move detailed or conditional guidance into supporting files, link each file clearly, and state exactly when to read it.*

Exercise 5: A Manual-Only Skill (3 min)

Finish with the deterministic user-invoked pattern. Use a short description that stays on one YAML line:

```
Create .claude/skills/ship-checklist/SKILL.md with EXACTLY this frontmatter:
---
name: ship-checklist
description: "Run the project release checklist only when the user selects /ship-checklist."
disable-model-invocation: true
allowed-tools: Read, Grep, Glob, Bash
---

Workflow:
1. Run npm test
2. Check source files for console.log
3. Verify package.json has a version
4. Verify .env.example exists
Report PASS/FAIL for each check and finish with SHIP or DO NOT SHIP.
```

Open `SKILL.md` in VS Code and confirm the frontmatter exactly matches the block above. Save and close the file before testing.

Open a new Claude Code tab/session, return to the same project, close the old tab, type `/`, and select:

```
/ship-checklist
```

Expected output marker: the `ship-checklist skill` invocation appears and the four checks return PASS/FAIL with a final SHIP or DO NOT SHIP verdict.

> `disable-model-invocation` *controls how the named skill is invoked; it is not a security boundary for Claude's general tools. Lab 2B uses deterministic hooks when an action must actually be controlled.*

Exercise 6: Tune Trigger Precision (5 min)

Predict whether the review skill should activate for each near-boundary prompt before running it:

Prompt	Expected
`Explain what @src/routes/tasks.js does`	Do not activate
`Refactor @src/routes/tasks.js for readability`	Do not activate
`Look for security risks in @src/routes/tasks.js`	Activate
`Summarize @package.json`	Do not activate
`Check this file @server.js to see if it is ready to commit`	Activate

If a case behaves incorrectly, edit **only the skill description**. Then rerun the failed case, one original

positive case, and the original implementation negative case.

Exercise done when: at least 4 of 5 cases behave correctly, an original review prompt still activates the skill, the implementation prompt still does not activate it, and the skill remains read-only.

The lesson is precision, not adding more trigger phrases: the description is a routing contract that must separate review intent from explanation, implementation, refactoring, and documentation intent.

VS Code Acceptance Gate and Process Receipt

Before commit, capture the same supervised workflow used across Day 2:

1. **Orient:** select the target range in the editor and use `Alt+K` or an explicit `@file#line-range`; explain unfamiliar code before editing.
2. **Plan:** review the bounded plan in the Claude Code panel; add an inline comment when the plan misses a constraint.
3. **Approve and implement:** approve only the requested scope and exact permissions.
4. **Inspect diff:** review every changed file in VS Code Source Control and reject unexpected files or authority changes.
5. **Verify:** clear the Problems panel for the touched scope and run the lab's deterministic tests or fixtures in the integrated terminal.
6. **Decide:** record the request, approved plan, changed files, verification evidence, and the human commit/install decision.

A model summary or grader result supports this receipt; it does not replace the diff review.

FINISH MARKER - Success Checklist

- [ ] `.claude/skills/dashboard-code-review/SKILL.md` with all five core controls
- [ ] 7+ trigger phrases + negative boundary in description
- [ ] During formal skill execution, `allowed-tools` pre-approves Read/Grep/Glob and `disallowed-tools` removes Write/Edit/Bash
- [ ] 4 of 5 activation tests behave correctly (incl. the negative test)
- [ ] references/javascript-security.md loads for JavaScript/TypeScript, but not for package.json
- [ ] ship-checklist frontmatter exactly matches the valid one-line YAML
- [ ] `/ship-checklist` explicitly runs four checks and returns the final verdict
- [ ] 4 of 5 Exercise 6 boundary cases behave correctly

If Stuck

Problem	Recovery
Skill never fires	Description too timid - add trigger phrases; be pushy
Skill fires on everything	Add "Do NOT use for..." boundaries
Skill file in the wrong place	Say "in the current project directory" - otherwise Claude may write to `~/.claude/` (global)

Problem	Recovery
New skill missing from the `/` menu	Ask Claude which project skills are loaded. If it lists the skill, refresh the session or VS Code extension to update the menu. If it does not, confirm `SKILL.md` is saved under the correct project directory, then open a new Claude Code tab/session.
No RED/YELLOW /BLUE format	The skill didn't activate - confirm Claude lists it as loaded, then check the description

Debrief Questions

1. What is the description field actually for - and who is its audience?
2. When do you set `disable-model-invocation: true`?
3. Which field constrains tools during formal skill execution: `allowed-tools` or `disallowed-tools` - and why is that not a project-wide security boundary?